KFN PHARMA — Node.js + PostgreSQL

Site internet + application de gestion · installation et mise en ligne

Site internet (slider 3D, services, activités, équipe, contact, recherche de produits) et application **Système des ventes et de gestion de stocks** (siège + agences), réécrits en **Node.js (Express)** avec une base **PostgreSQL**. Responsive : ordinateur, tablette et téléphone.

1. Règles de gestion

- **Les clients de l'AGENCE SIEGE sont les autres agences.** Au siège, la liste des clients est la liste des agences (on ne crée pas de client libre au siège ; une nouvelle agence devient automatiquement cliente du siège).
- **À la caisse du siège, seules les agences paient** : leurs factures d'approvisionnement et leurs achats au siège.
- **Chaque autre agence a ses propres clients**, qu'elle crée, facture et encaisse elle-même.
- **Circuit d'approvisionnement** : l'agence fait une pré-commande → le siège valide (quantité possible à prendre) → l'agence réceptionne. À la réception, une **facture du siège au nom de l'agence** est émise automatiquement ; l'agence la règle à la caisse du siège ou par versement (*Agence Siège → Compte des agences*). Une quantité refusée peut être **retournée au siège**.
- **Vente directe du siège à une agence** (*Ventes → Édition de factures*, agence = AGENCE SIEGE) : les articles passent du stock du siège au stock de l'agence, et la facture est à régler à la caisse du siège.

2. Workflow des pré-commandes et notifications WhatsApp

1. **Lancement** — une agence enregistre une pré-commande : le siège la reçoit automatiquement dans l'application (*Agence Siège → Approvisionnement des agences*, compteur sur l'accueil) **et** reçoit un message WhatsApp avec le détail de la commande (n°, agence, articles, quantités, montants, total).
2. **Validation** — quand le siège valide, l'agence reçoit un message WhatsApp avec les quantités accordées article par article (signalées quand l'accordé est inférieur au demandé) et le montant accordé.
3. **Réception** — quand l'agence accepte les produits, et/ou en retourne au siège, le siège reçoit un message WhatsApp listant les produits acceptés, les produits refusés, la facture du siège émise et ce qu'il reste à traiter.

Paramétrage : ADMIN → Notifications WhatsApp

- **Numéros** : numéros du siège (plusieurs possibles) et numéro(s) de chaque agence (aussi modifiables dans *Paramétrage des agences*). Les numéros ivoiriens à 10 chiffres sont acceptés tels quels (indicatif 225 ajouté automatiquement).

- **Service d'envoi** au choix :

Service	À savoir
WhatsApp Cloud API (Meta)	Officiel. Jeton permanent + identifiant du numéro. Pour écrire à tout moment, créer un modèle « Utility » (ex. `kfn_notification`, corps `KFN PHARMA : {{1}}`)
Twilio	Account SID, Auth Token, numéro WhatsApp Twilio
UltraMsg	Utilise un numéro WhatsApp existant (QR code) ; non officiel : prendre un numéro dédié
Webhook HTTP	Envoie `{to, telephone, message}` en JSON à l'adresse de votre choix (autre passerelle, n8n, Make…)
Aucun	Les messages sont préparés dans le journal avec un bouton « Ouvrir dans WhatsApp » pour les envoyer à la main

- **Messages à envoyer** : chaque étape peut être activée ou désactivée.
- **Test** : envoi d'un message de test à un numéro.
- **Journal** : historique de tous les messages (envoyé, en échec avec la raison, à envoyer manuellement), bouton « Renvoyer ».
- **Adresse de l'application** : si elle est renseignée, un lien direct vers l'écran à traiter est ajouté aux messages.

3. Prérequis

- **Node.js 18 ou plus** (conseillé : 22 LTS) — https://nodejs.org
- **PostgreSQL 12 à 17** (local, ou base en ligne : Render, Neon, Supabase, Railway…)

4. Installation en local (Windows, macOS, Linux)

1. Installez Node.js et PostgreSQL. Avec pgAdmin ou `psql`, créez une base vide : `CREATE DATABASE kfnpharma;`
2. Décompressez le projet, ouvrez un terminal dans le dossier `kfnpharma-node`, puis tapez `npm install` et `copy .env.exemple .env` (macOS / Linux : `cp .env.exemple .env`).
3. Ouvrez `.env` et renseignez l'accès à PostgreSQL (`PGHOST`, `PGDATABASE`, `PGUSER`, `PGPASSWORD`) ou une `DATABASE_URL`.
4. Lancez : `npm start` → **les 37 tables et les données de départ sont créées automatiquement** au premier démarrage.
5. Ouvrez **http://localhost:3000** (site) et **http://localhost:3000/gestion** (application).

Comptes de départ (mots de passe chiffrés automatiquement à la 1re connexion — **changez-les aussitôt** dans *ADMIN → Utilisateurs*) :

- **superadmin / Super@2026** — Super Admin
- **admin / Admin@2026** — Administrateur

Commandes utiles : `npm run db:init` (crée les tables si absentes), `npm run db:reset` (**efface tout** et recrée), `npm run dev` (redémarrage automatique pendant le développement).

5. Mise en ligne

Important : InfinityFree n'exécute pas Node.js (seulement PHP). Pour cette version, choisissez un hébergeur compatible Node.js ; la version PHP/MySQL (KFNPHARMA_InfinityFree.zip) reste disponible pour InfinityFree.

Render (render.com) — le plus simple, avec offre gratuite

1. Mettez le dossier du projet sur GitHub (sans `node_modules` ni `.env`).
2. Render → **New → Blueprint** → choisissez le dépôt : le fichier `render.yaml` crée le service web **et** la base PostgreSQL.
3. Au premier démarrage, les tables sont créées automatiquement. Ouvrez l'adresse `https://…onrender.com`. Vérifiez les conditions actuelles de l'offre gratuite (mise en veille du service, durée de vie de la base gratuite) : pour une utilisation quotidienne par toutes les agences, prenez une offre payante.

Railway, Fly.io, Koyeb… : créez un service Node.js (commande `npm start`) et une base PostgreSQL, puis définissez `DATABASE_URL`, `SESSION_SECRET` et `SECURE_COOKIES=true` dans les variables d'environnement.

Base en ligne Neon / Supabase : copiez leur chaîne de connexion dans `DATABASE_URL` (elle contient `sslmode=require`).

Serveur VPS (Ubuntu) : `docker compose up -d` (fichiers `Dockerfile` et `docker-compose.yml` fournis), ou bien Node.js + PostgreSQL installés, `npm ci --omit=dev`, puis `pm2 start server.js --name kfnpharma` derrière Nginx avec HTTPS (Let's Encrypt).

6. Variables d'environnement (`.env`)

Variable	Rôle
`PORT`	Port HTTP (3000 par défaut)
`DATABASE_URL`	Connexion PostgreSQL complète (prioritaire)
`PGHOST`, `PGPORT`, `PGDATABASE`, `PGUSER`, `PGPASSWORD`	Connexion PostgreSQL détaillée
`PGSSL=true`	Base distante qui exige SSL
`SESSION_SECRET`	Phrase secrète des sessions — **à changer**
`SECURE_COOKIES=true`	À activer quand le site est en HTTPS

7. Arborescence

```
server.js                serveur Express (site, connexion, routeur de l'application)
src/config.js            lecture du .env
src/core.js              PostgreSQL, utilitaires, profils et droits, numérotation, stock
src/web.js               menus, droits par page, mise en page
src/site.js              site internet (accueil, contact, recherche)
src/ventes.js            factures, transfert siège → agence, facture d'appro
src/api.js · print.js    fenêtres de choix (JSON) · impressions
src/tables.js            métadonnées des tables (éditeur, import / export Excel)
src/pages/*.js           un fichier par écran de l'application
public/assets/           styles, scripts, images
database/schema.sql      structure PostgreSQL (37 tables)
database/seed.sql        données de départ
```

8. Profils

Profil	Code	Accès
Super Admin	SUP	Tout, toutes les agences
Administrateur	ADM	Tout, sauf modifier / supprimer dans *ADMIN → Modification des fichiers*
Responsable d'agence	RAG	Caisse + ventes + consultation + stock de son agence
Gestionnaire stock	GST	Uniquement le stock
Caissier(ère)	CAI	Caisse + consultation
Vendeur(se)	VEN	Ventes + consultation
Consultation	CON	États (au siège : aussi les statistiques de toutes les agences)

9. Fonctions

Toutes les fonctions de la version précédente sont reprises : familles, articles, stocks et prix par agence, approvisionnement du dépôt principal, circuit pré-commande → validation → réception avec retour au siège, point des appros par année, factures et pro-forma, modification, caisse (jour, impayés, dépenses, fermeture, arrêts), 12 états avec dates de début et de fin, **Statistique de vente par agence** (classement, graphiques), onglet **ADMIN** (utilisateurs, paramétrage des agences, modification des fichiers façon Excel, importation / exportation Excel avec modèles à remplir, import CSV), questions de confirmation (« Voulez-vous valider l'enregistrement ? », « Voulez-vous modifier cette opération ? », « Voulez-vous supprimer ? Attention ! … ») et messages « Enregistrement / Modification / Suppression réussi(e) ».

10. Base de données PostgreSQL

Les noms des tables et des colonnes de l'analyse WinDev sont conservés, **en minuscules** (PostgreSQL ne distingue pas la casse sans guillemets) : `T_Stock` → `t_stock`, `RefArticles` → `refarticles` … Les fichiers Excel de l'ancienne version peuvent être importés : les noms de colonnes sont reconnus sans tenir compte des majuscules. Sauvegarde : *ADMIN → Exportation des données (Excel)* et/ou `pg_dump kfnpharma > sauvegarde.sql`.